Group 6 - Budget Buddy (A Finance Tracking WebApp)
FINAL REPORT

Team members

Name and Student id	**GitHub id**	Number of Story points that member was an **author on**.
Jasleen Minhas, 202481225	**https://github.com/JasleenMinhas578**	198
Masroor Rahman, 202482239	https://github.com/MshRoor	147
Sumaiya Khan, 202480995	https://github.com/sumaiyak187	104
Kaustubh Patil, 202580621	https://github.com/KaustubhPatil25	95
Ronit Gajjar, 202488048	https://github.com/HELLOMOTO199	95
Joel George Sam, 202483190	https://github.com/joelgeorgesam	92

Project summary:

Budget Buddy is a free, easy-to-use web application that helps users track personal expenses, visualise spending, and export reports without paywalls or subscriptions. Built with React and Firebase (Auth + Firestore), it provides secure authentication, real-time expense and category management (CRUD), responsive dashboards with Pie/Bar/Line charts, and one-click PDF/CSV report generation. Quality is enforced through a fully automated CI/CD pipeline (GitHub Actions + Vercel), 295 Jest unit tests, 102 Cypress E2E tests, 100% coverage, zero ESLint issues, and Lighthouse scores of 99% performance and 100% accessibility.

Velocity and a list of user stories and non-story tasks for each iteration:

Over the lifetime of the project, the team completed **53 issues** (29 feature user story issues and 24 infrastructure/support issues), for a total of **211 story points across 5 iterations (≈10 weeks)**.

Average Velocity was **~42 SP** per Iteration.

Iteration & Focus	**Issues**	**Story Points**	**Key Feature Stories**	**Key Infrastructure Stories**	**GitHub Issues Links**
Iteration 1 – Setup & Planning	8	35	- Initial Setup and Planning	- Requirements gathering - Team roles & issue creation - Repo and workflow setup - Initial architecture & UML - React app initialization - Firebase setup & config.	#1, #2, #3, #4, #5, #6, #7, #16
Iteration 2 – Auth & Landing Experience	20	76	- User Registration (4 issues, 16 SP) [Status: Done] - User Login (5 issues, 18 SP) [Status: Done] - User Logout (2 issues, 5 SP) [Status: Done]	- Landing page & navigation - Auth form validation and error handling - Unit tests for auth - CI/CD pipeline - Initial Vercel deployment - Styling and responsive layout.	#8, #9, #10, #11, #12, #13, #36, #38, #40, #42, #43, #44, #45, #46, #47, #49, #50, #58, #59, #62
Iteration 3 – Dashboard, Expenses, Categories	13	46	- View Dashboard (4 issues, 18 SP) [Status: Done]	- Firestore wiring for expenses/categories - Dashboard widgets - UI/UX improvements - Bug fixes	#20, #21, #22, #23, #24, #25, #26, #28, #29, #66, #78, #81, #85

			- Manage Expenses (5 issues, 21 SP) [Status: Done] - Manage Categories (5 issues, 16 SP) [Status: Done]	- Documentation & README updates - CI reliability tweaks.	
Iteration 4 – Reporting & Visualizations	7	28	- Generate Reports (4 issues, 19 SP) [Status: Done]	- Chart integration and configuration - Dashboard layout refinement - Expense summary widgets - Unit tests for dashboard - Forgot-password/password-reset implementation.	#30, #31, #32, #33, #34, #35, #87
Iteration 5 – Testing, Quality & Finalization	5	26	- No new key feature stories, just correcting existing features and Testing	- Cypress E2E tests - Cross-browser checks - Accessibility & performance audits (e.g., Lighthouse) - ESLint cleanup - Final release checklist and regression test coverage.	#91, #94, #98, #100, #102
	Total Issues: 53	**Total SP: 211**	**Key Feature Issues: 29**	**Infrastructure Issues: 24**	

For detailed information about the issues, story points and planning, visit: "Planning_Mapping.md"
For detailed understanding of the issues with acceptance criteria, visit: "Acceptance_Tests.md"

Overall Arch and Design

Budget Buddy is implemented using a **client–server architecture** with a **layered single-page application (SPA)** on the client side and **serverless backend services** on the cloud.

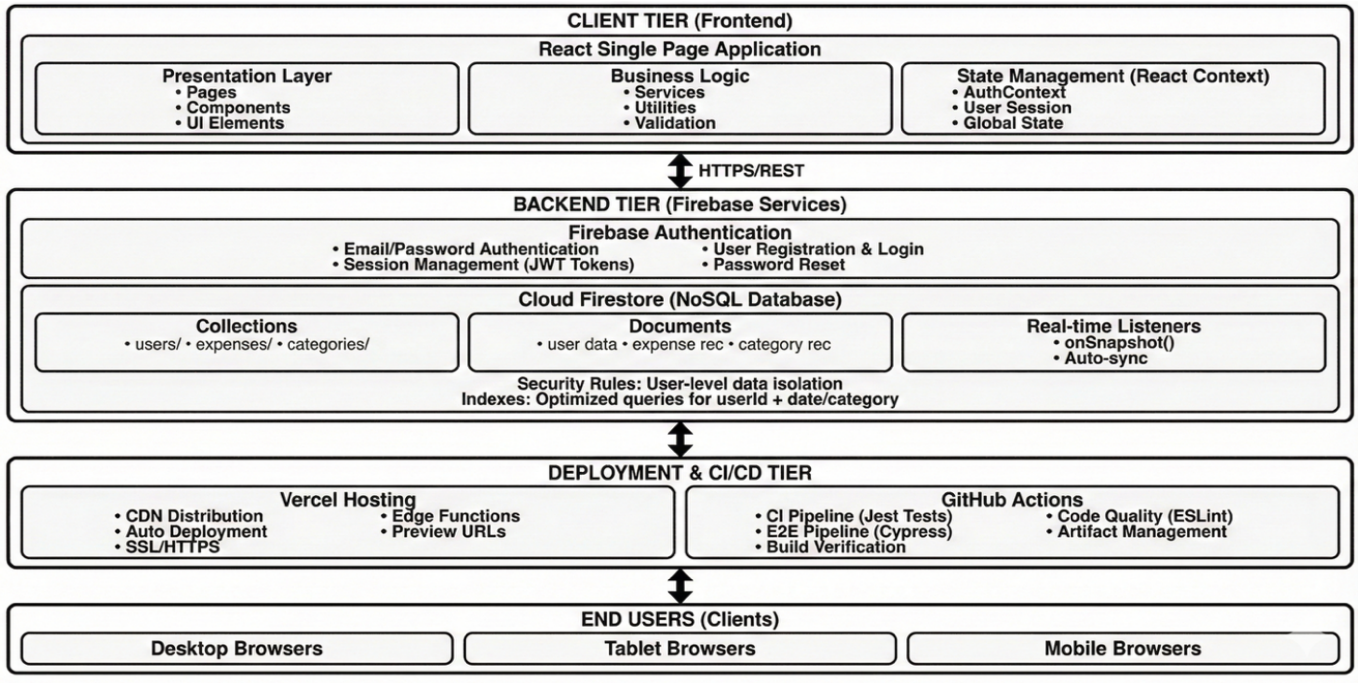

Figure 1. High Level System Architecture of Budget Buddy

On the client side, a React application runs entirely in the user's browser and is deployed as optimized static assets on Vercel. Internally, the client follows a layered structure:

- **Presentation layer**: React pages and components for authentication (login, signup), dashboard, expenses, categories, and reports, along with chart components for visualising data.
- **Business logic layer**: Contexts and service modules implement validation, orchestration of user flows (e.g., add expense → recalculate dashboard statistics), and encapsulate core application rules.
- **Data access layer**: A thin wrapper around the Firebase SDK provides CRUD operations on Cloud Firestore and exposes authentication operations (signup, login, logout) via Firebase Auth.

On the backend, **Firebase Authentication** handles secure email/password login, account creation, and session management, while **Cloud Firestore** stores all user-specific expenses and categories. Each document is keyed by a userId, and Firestore Security Rules ensure that users can only access their own data. The system relies on Firestore's onSnapshot listeners to provide **real-time updates**, so that any changes to expenses or categories are immediately reflected on the dashboard without manual refresh.

The built React application is deployed to **Vercel's global CDN**, which serves users on desktop, tablet, and mobile. **GitHub Actions** runs automated pipelines (linting, unit tests, end-to-end tests) on each push, and successful builds are automatically deployed to Vercel. This setup provides a small but complete client–server system with clear separation of concerns and an automated path from code to production.

To document the design, we use the **4+1 architectural view model** with standard UML diagrams.

- The **Logical View** is captured with a Class Diagram and Component Diagram that model core entities (User, Expense, Category, CategoryStats, Report) and their relationships, as well as main UI components (Auth, Dashboard, Expense, Charts).
- The **Process View** is represented using sequence and state diagrams for key flows such as user authentication and expense lifecycle.
- The **Development View** shows the source-code structure (components, context, services, tests).
- The **Physical View** is described with a network/deployment diagram of Vercel edge nodes communicating with Firebase Auth and Firestore.
- Finally, the **Scenarios (Use Case View)** use UML use-case diagrams to tie together the different views from an end-user perspective.

To Show applicable parts (at least one) from the 4+1 logical/physical/etc. model, with the appropriate UML techniques we covered, we have represented with the **Logical View with Class Diagram.**

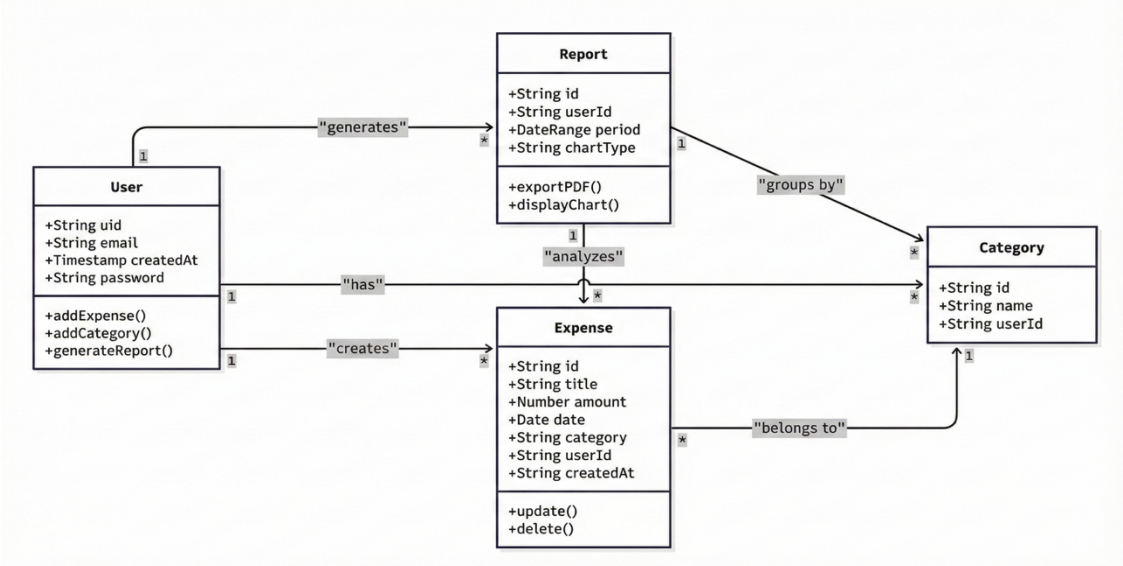

Figure 2. Logical View using Class Diagram

For a detailed description of the System Architecture of Budget Buddy, visit "Architecture_Diagrams.md"

Infrastructure & Technology Choices (Summary):

Frontend technology:

- **React.js (**Link: https://react.dev)
  - **Why do we use it:**
    React's component-based model and strong ecosystem let us quickly build a responsive single-page app with reusable UI elements and predictable state handling, which fits the project's timeline and team skills.
  - **Alternatives and why they don't work:**
    Angular is heavier and has a steeper learning curve for a small student team; Vue has a smaller ecosystem and fewer Firebase-focused resources. React offered the simplest, most familiar path for us.

Backend technology:

- **Firebase Authentication (**Link: https://firebase.google.com/products/auth)
  - **Why do we use it:**
    Firebase Auth provides secure, production-ready login, registration, and password reset out of the box, integrating smoothly with React and Firestore while minimising the security work we would otherwise have to implement ourselves.
  - **Alternatives and why they don't work:**
    Auth0 and AWS Cognito add extra setup and potential cost for a simple student project, while building custom authentication would be error-prone and time-consuming.
- **Cloud Firestore (**Link: https://firebase.google.com/docs/firestore**)**
  - **Why do we use it:**
    Firestore gives a serverless NoSQL document store with real-time updates, automatic scaling, and tight integration with Firebase Auth, which maps naturally to per-user budgets and expense documents.
  - **Alternatives and why they don't work:**
    Relational databases (e.g., PostgreSQL) or MongoDB Atlas require additional backend services and don't provide built-in real-time listeners; Firebase Realtime Database is less flexible in querying and structure for our reporting needs.

Deployment technology:

- **Vercel (**Link: https://vercel.com**)**
  - **Why do we use it:**
    Vercel lets us deploy the React app directly from GitHub with almost no configuration, providing automatic previews, global CDN, HTTPS, and scaling—ideal for a course project with limited DevOps time.
  - **Alternatives and why they don't work:**
    Netlify and AWS Amplify are viable but require more configuration for our specific stack, while traditional hosting adds server management overhead that doesn't add value here.
- **GitHub Actions (**Link: https://github.com/features/actions)
  - **Why do we use it:**
    GitHub Actions is built into our repository, so we can define simple YAML workflows for automated tests and deployments without extra services, keeping CI/CD close to our code and version control.
  - **Alternatives and why they don't work:**
    Jenkins, CircleCI, and Travis CI require separate setup, hosting, or integration; for a small academic project, that extra complexity is unnecessary compared to GitHub's native solution.

For more details about the Infrastructure and Technology choices, visit "README.md"

Name Conventions:

We followed a consistent, project-wide naming convention aligned with common JavaScript/React standards (documented in Naming_Conventions_Summary.md):

- **Files & Components:** React components use **PascalCase** (e.g., DashboardPage.tsx, ExpenseForm.tsx), while utility files use **camelCase** (e.g., dateUtils.ts).
- **Variables & Functions:** Use **camelCase** for all variables, functions, hooks, and constants (e.g., handleSubmit, totalExpenses, useAuthContext).
- **CSS / Styles:** Use **kebab-case** for CSS files and class names (e.g., dashboard-container, expense-row).
- **Firebase & Data:** Firestore collections are **lowercase plural** (e.g., users, expenses, categories), with document fields in **camelCase**.
- **Tests & Config:** Test files mirror the component name with .test/.spec suffix (e.g., ExpenseForm.test.tsx), and config files follow standard tool-specific names (e.g., eslint.config.js, tsconfig.json).

These conventions improve readability, make the codebase easier to navigate as a team, and align with widely used online JavaScript/React naming standards.

Code:

Top 5 most important files (full path):

File path with a clickable GitHub link	Purpose
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/services/database.js	This file provides all Firestore CRUD and real-time subscription functions for managing user-specific expenses and categories in the app.
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/context/AuthContext.js	This file creates a global authentication context that manages Firebase user login, signup, logout, password resets, and exposes the current user across the app.
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/components/Expense/ExpenseList.jsx	This file renders the Expense List component, providing real-time expense fetching, filtering, sorting, and deletion with Firebase integration and UI animations.
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/components/Auth/Login.jsx	This file implements the Login component, handling user authentication with Firebase, form validation, error feedback, and redirecting users to the dashboard upon successful login.
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/components/Layout/PrivateRoute.jsx	This file defines a Private Route component that protects routes by checking Firebase authentication, showing a loading state, and redirecting unauthenticated users to login while preserving their intended destination.

Testing and Continuous Integration:

Top 5 most important unit tests with links below:

Test File path with a clickable GitHub link	What is it testing
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/__tests__/ExpenseForm.test.jsx	It tests the Expense Form component's form behavior, validation, category loading, add/edit submissions, error states, and interaction handling using mocked Firebase and callback functions.
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/__tests__/Login.test.jsx	It tests the Login component's authentication flow, UI behavior, and error handling by mocking Firebase and verifying navigation, validation, and user interactions.
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/__tests__/TestFirebase.test.jsx	It tests the Test Firebase component's Firebase authentication flows (anonymous and email/password), Firestore read/write operations, error handling, and sign-out functionality using mocked Firebase services.

https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/__tests__/Expenses.test.jsx	It tests the Expenses dashboard component's rendering, summary calculations, modal and expense form integration, Firebase Firestore data fetching/listening, and error handling using mocked services.
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/src/__tests__/DashboardOverview.test.jsx	It tests the Dashboard Overview component's rendering, summary calculations, recent expenses display, sorting, empty states, and Firebase integration using mocked authentication and Firestore data.

For detailed Unit test coverage and test catalogue, visit Jest_Unit_Testing folder. All the unit tests are written in the __test__ folder. The Unit test has 100% coverage. View the entire report here.

Top 5 most important acceptance tests with links below:

Test File path with a clickable GitHub link	Which user story is it testing
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/cypress/e2e/04-expenses.cy.js	It tests the full user story that an authenticated user can add, view, and manage expenses, with proper form behavior and persistent data in the Budget Buddy app. The user stories links: #20, #21, #22, #23, #24
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/cypress/e2e/01-signup.cy.js	It tests the complete user story that a new user can successfully sign up, with full validation, error handling, navigation, and UX behaviors throughout the registration flow. The user stories links: #10, #45, #46, #47
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/cypress/e2e/02-login.cy.js	It tests the complete user story that an existing user can securely log in, with proper validation, error handling, session persistence, UX behaviors, and navigation between authentication pages. The user stories link: #9, #42, #43, #44, #87
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/cypress/e2e/03-dashboard.cy.js	It tests that the authenticated user's dashboard displays correctly, shows summaries and navigation, allows quick actions, remains responsive, and maintains session persistence. The user stories link: #30, #31, #33, #58
https://github.com/JasleenMinhas578/BudgetBuddy/blob/main/cypress/e2e/06-reports.cy.js	It tests that users can access, view, filter, and export financial reports with correct data, charts, summaries, and responsive layout. The user stories link: #30, #32, #34, #35

For detailed information about Acceptance Tests, visit "Acceptance_Tests.md". All the Acceptance tests are written in Cypress under "cypress/e2e" folder. We also did Beta Acceptance Testing, and the feedback is present in the "Beta Acceptance Testing section of Readme.md"

Continuous Integration Environment

Our project uses **GitHub Actions** as the continuous integration (CI) environment. Every push and pull request to the main branches triggers an automated workflow that checks the health of the codebase before changes are merged. The workflow installs dependencies, runs static analysis with **ESLint**, executes **unit tests** (Jest/React Testing Library) and **Cypress end-to-end tests**, and then builds the production React app. CI runs directly inside GitHub, so logs, artefacts, and test results are visible alongside each commit and pull request.

If the pipeline fails at any stage (linting, tests, or build), the pull request is blocked until the issue is fixed. When the pipeline passes on the main branch, the build is eligible for Continuous deployment (CD) to **Vercel**, ensuring that only code that has passed all automated checks reaches the live environment.

For more detailed information about the CI/CD pipeline, visit "CI/CD section of Readme.md".

To see the status of CI and CD, visit "Github Actions" and "Deployments" on GitHub.

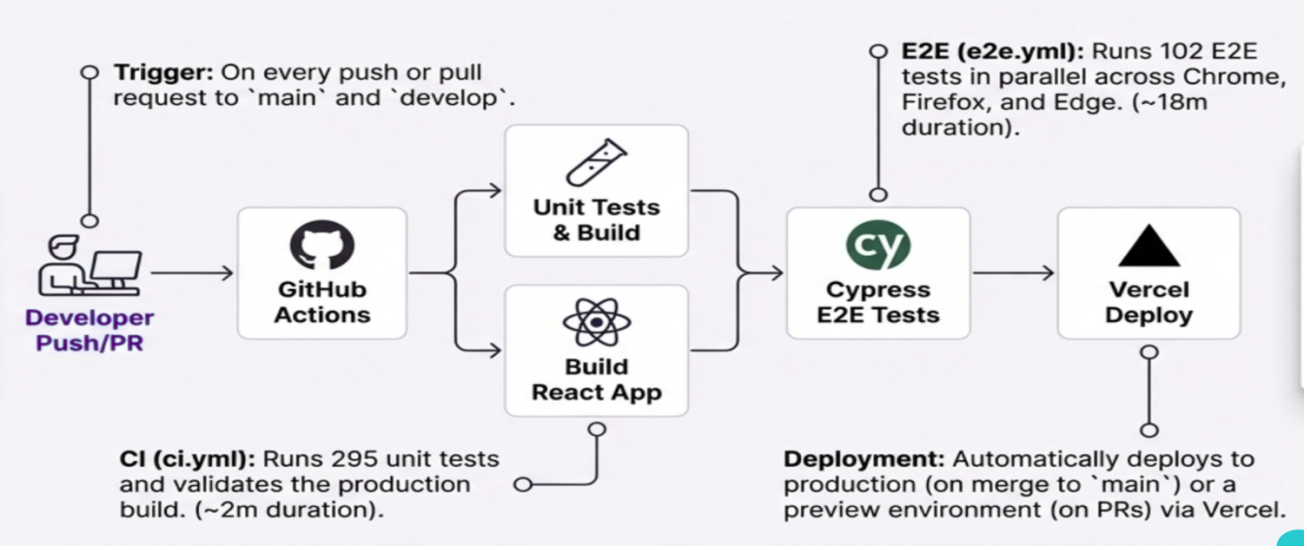

Figure 3. CI/CD Pipeline

Static Analysis tool

We use **ESLint** as our static analysis tool because most of our codebase is **JavaScript React**. ESLint is the standard linter for JS/TS, with first-class support for React and JavaScript via plugins, so it can catch unused variables, type and import issues, unsafe patterns, and common bugs directly in our .jsx and .js files. Using a single, configurable ruleset keeps the whole team aligned on style and quality and helps prevent problems early, before they show up in runtime behaviour or tests.

ESLint is run in two ways: **locally** and in **CI**. Locally, developers run it via an npm script (e.g., npm run lint) during development to get quick feedback in their editor/terminal and optionally use --fix for simple automatic corrections. The same command is executed in our **GitHub Actions** pipeline on every push and pull request; if ESLint reports any errors, the CI job fails and the pull request cannot be merged. This ensures that all code merged into the main branch has passed static analysis for the primary languages used in the project.

For more detailed information about the Static analysis report, visit "ESLint_Report.md"

Appendix

Resource	Link
Live Application	budget-buddy-mun.vercel.app
CI/CD Results	GitHub Actions
Deployments	Vercel Deployments
Planning Board	GitHub Projects
Requirements Document	Documents/Requirements.md
Planning & Issues Mapping	Documents/Planning_Mapping.md
Static Analysis (ESLint) Report	eslint-report/ESLint_Report.md
Performance (Lighthouse) Report	Documents/Lighthouse_Metrics/
Acceptance Tests & Requirements	Documents/Acceptance_Tests.md
E2E Test Docs	Documents/Cypress_E2E_Testing/
Unit Test Docs	Documents/Jest_Unit_Testing/
Beta Acceptance Testing	Beta testing Results
Naming Convention Summary	Documents/Naming_Convention_Summary.md
Running Instructions	README.md - How to run section
Demo Video	Google Drive Link - Video